61. Структуры

В программировании есть много случаев, когда может понадобиться больше одной переменной для представления определенного объекта.

Оглавление:

1. Зачем нужны структуры?
2. Объявление и определение структур
3. Доступ к членам структур
4. Инициализация структур
5. C++11/14: Инициализация агрегатных членов структур
6. Присваивание значений членам структур
7. Структуры и функции
8. Вложенные структуры
9. Размер структуры
10. Доступ к структурам из нескольких файлов
11. Заключение
12. Тест

Зачем нужны структуры?

Например, для представления книги (title, author, price), заостря разность такой книги, дата, рекламная, доп. все или любую другую информацию:

```
book1.title = "Learn C++";
book1.author = "Alex";
book1.price = 19.95;

book2.title = "Hello, World!";
book2.author = "John";
book2.price = 39.95;

std::cout << "Title of first book: " << book1.title << '\n';
std::cout << "Author of first book: " << book1.author << '\n';
std::cout << "Price of first book: " << book1.price << '\n';

std::cout << "Title of second book: " << book2.title << '\n';
std::cout << "Author of second book: " << book2.author << '\n';
std::cout << "Price of second book: " << book2.price << '\n';
```

Что потребует 3 для всех 6 переменных. Если вы захотите передать информацию в себе в функцию, то вам придется передавать каждую переменную по отдельности. Кроме того, если вы захотите изменить информацию о книге под, то вам придется дублировать объявление ещё 3 переменных на каждую переменную сложно видно, что такая реализация не очень эффективна.

В частности, можно сказать, что появляется проблемы с сущностью, называемой **членами структуры** или **членами структуры**. Переменные — это просто объявленные структуры. Хотя вы и решаете компилятору, что вам нужно переменные помечены, память под них выделена, но выделена. Именно структуры помогают решить с вышеописанной проблемы, чтобы отличить их от меню переменных.

Предупреждение: Одна из самых простых ошибок в C++ — забыть точку с запятой в конце объявления структуры. Это приводит к ошибке компиляции в следующей строке кода. Современные компиляторы, такие как Visual Studio версии 2010, а также более новые версии, укажут вам, что вы забыли точку с запятой в конце, но более старые компиляторы могут этого и не сделать, из-за чего вы можете будет подвергнуться.

Чтобы использовать структуру `Employee`, нам нужно просто объявить переменную типа `Employee`:

```
Employee fred;
Employee mike;
```

Здесь мы определили переменные типа `Employee` с именем `fred`. Как и в случае с обычными переменными, определение переменной, типа структуры приводит к выделению памяти для этой переменной.

Объявить можно и несколько переменных одной структуры:

```
Employee fred;
Employee mike;
```

Доступ к членам структур

Когда мы объявляем переменную структуры, например, `Employee fred`, то `fred` сохраняется как структура. Для того, чтобы получить доступ к отдельным членам структуры, мы используем оператор выбора членов (.). Например, в коде, приведенном ниже, мы используем оператор выбора членов для инициализации каждого члена структуры:

```
Employee fred;
fred.id = 5;
fred.wage = 23.45;

Employee mike;
mike.id = 6;
mike.wage = 34.67;

std::cout << "fred's id: " << fred.id << '\n';
std::cout << "fred's wage: " << fred.wage << '\n';

std::cout << "mike's id: " << mike.id << '\n';
std::cout << "mike's wage: " << mike.wage << '\n';
```

Как и в случае с обычными переменными, переменные члены структуры не инициализируются автоматически и обычно приводят к ошибке. Инициализировать их нужно вручную.

В приведенном выше коде мы определили шесть переменных относится к структуре `Employee`, а также к структуре `James`. Это обеспечивает гораздо более высокий уровень организации, чем в случае с обычными отдельными переменными.

Переменные члены структуры работают так же, как и простые переменные, поэтому с ними можно выполнять любые арифметические операции и операции присваивания:

```
Employee harry = { 14, "Harry", 30.25 };
Employee larry = { 15, "Larry", 20.25 };

int id = harry.id + larry.id;
int wage = harry.wage + larry.wage;

std::cout << "Combined id: " << id << '\n';
std::cout << "Combined wage: " << wage << '\n';

harry.id = 16;
larry.wage = larry.wage * 1.5;

std::cout << "Harry's new id: " << harry.id << '\n';
std::cout << "Larry's new wage: " << larry.wage << '\n';
```

Инициализация структур

Инициализация структур почти аналогична значений каждому члену по порядку — значение данных суммарно (особенно, если это членов много), поэтому в языке C11 есть более быстрый способ инициализации структур — с помощью **списков** (также называется). Это позволяет инициализировать некоторые или все члены структуры при одном объявлении переменной типа struct:

```
struct Employee
{
    int id = 0;
    int age = 0;
    double salary = 0.0;
};

Employee joe = { 1, 32, 3500.0 }; // joe.id = 1, joe.age = 32, joe.salary = 3500.0
Employee jim = { 2, 28, 4500.0 }; // jim.id = 2, jim.age = 28, jim.salary = 4500.0
```

В C11 также можно использовать **шаблон инициализации**:

```
Employee joe { 1, 32, 3500.0 }; // joe.id = 1, joe.age = 32, joe.salary = 3500.0
Employee jim { 2, 28, 4500.0 }; // jim.id = 2, jim.age = 28, jim.salary = 4500.0
```

Если в списке инициализации задано меньше значений, чем членов структуры, то остальные будут инициализированы по умолчанию (обычно). Во 3 примере, приведенном выше, члену `jim.salary` присваивается значение по умолчанию. **И.Е.** так как мы сам не предоставили значение для инициализации, и оно можно можно использовать.

C++11/14: Инициализация агрегатных членов структур

В C11 добавили возможность присваивать нестатические обычные членам структуры значения по умолчанию. Например:

```
struct Triangle
{
    double length = 1.0;
    double width = 1.0;
};

int main()
{
    Triangle t1 { 2.0 };
    Triangle t2 { 3.0, 4.0 };

    std::cout << "t1 length: " << t1.length << '\n';
    std::cout << "t1 width: " << t1.width << '\n';

    std::cout << "t2 length: " << t2.length << '\n';
    std::cout << "t2 width: " << t2.width << '\n';

    return 0;
}
```

Следовательно, если кроме того, вы предоставляете инициализацию нестатических членов для любой инициализацию, шаблон инициализации более гибкие, поэтому рекомендуется использовать именно ее.

Однако в C++14 его ограничения были ослаблены, и обе версии можно использовать. Мы еще поговорим детальнее о статических членах структуры на соответствующем уроке.

Присваивание значений членам структур

До C++11, если бы мы хотели присвоить значения членам структуры, то нам бы пришлось это делать вручную каждому члену по отдельности:

```
struct Employee
{
    int id = 0;
    int age = 0;
    double salary = 0.0;
};

Employee joe;
joe.id = 1;
joe.age = 32;
joe.salary = 3500.0;

Employee jim;
jim.id = 2;
jim.age = 28;
jim.salary = 4500.0;
```

Это весь особенно когда членов в структуре много. В C++11 вы можете присвоить значения членам структуры, используя список инициализации:

```
struct Employee
{
    int id = 0;
    int age = 0;
    double salary = 0.0;
};

Employee joe;
joe = { 1, 32, 3500.0 };

Employee jim;
jim = { 2, 28, 4500.0 };
```

Структуры и функции

Большим преимуществом использования структур, нежели отдельных переменных, является возможность передать все структуры в функции, которая должна работать с ней основной:

```
struct Employee
{
    int id = 0;
    int age = 0;
    double wage = 0.0;
};

void printInfo(Employee pEmployee)
{
    std::cout << "ID: " << pEmployee.id << "\n";
    std::cout << "Age: " << pEmployee.age << "\n";
    std::cout << "Wage: " << pEmployee.wage << "\n";
}

int main()
{
    Employee joe { 1, 32, 20.25 };
    Employee mike { 2, 28, 25.00 };

    // Print out the employees
    printInfo(joe);
    std::cout << "\n";
    printInfo(mike);

    return 0;
}
```

В примере, приведенном выше, мы передаем структуру `Employee` в функцию `printInformation()`. Это позволяет нам не передавать каждую переменную по отдельности. Вместо того, если мы всегда хотим заполнить добавить новые членов в структуру `Employee`, то нам не придется изменить объявление или вызов функции!

Результат выполнения программы:

```
ID: 1
Age: 32
Wage: 20.45

ID: 2
Age: 28
Wage: 25.00
```

Функция также может возвращать структуру (это одно из тех немногих случаев, когда функция может возвращать несколько переменных). Например:

```
struct Rectangle
{
    double m_l;
    double m_w;
    double m_a;
};

Rectangle getSomeRect()
{
    Rectangle temp = { 2.0, 3.0, 6.0 };
    return temp;
}

int main()
{
    Rectangle rect = getSomeRect();

    // Print out the area
    std::cout << "The area is: " << rect.m_a << '\n';

    return 0;
}
```

Результат выполнения программы:

```
The point is: 1
```

Вложенные структуры

Одна структура могут содержать другие структуры. Например:

```
struct Point
{
    int x = 0;
    int y = 0;
    int z = 0;
};

struct Company
{
    int id = 0;
    int age = 0;
    int wage = 0;
};

struct Company
{
    int id = 0;
    int age = 0;
    int wage = 0;
    int salary = 0;
    int hours = 0;
};

int main()
{
    Company company1 { 1, 32, 15, 0, 0 };
    Company company2 { 2, 28, 20, 0, 0 };

    // Print out the area
    printInfo(company1);
    std::cout << "\n";
    printInfo(company2);

    return 0;
}
```

Результат выполнения программы:

```
The point is: 1
```

Вложенные структуры

Одна структура могут содержать другие структуры. Например:

```
struct Point
{
    int x = 0;
    int y = 0;
    int z = 0;
};

struct Company
{
    int id = 0;
    int age = 0;
    int wage = 0;
    int salary = 0;
    int hours = 0;
};

int main()
{
    Company company1 { 1, 32, 15, 0, 0 };
    Company company2 { 2, 28, 20, 0, 0 };

    // Print out the area
    printInfo(company1);
    std::cout << "\n";
    printInfo(company2);

    return 0;
}
```

В этом случае, если вы вы хотели узнать, какая зарплата у CEO (использовалось директор), то нам бы пришлось использовать оператор выбора членов дважды:

```
Employee ceo{ 1, 42, 60.0 };
std::cout << ceo.boss.wage << '\n';
```

Согласно мы выбираем поле `CEO` из структуры `myCompany`, а затем поле `salary` из структуры `Employee`.

Вы можете использовать вложенные списки инициализации с вложенными структурами:

```
struct Employee
{
    int id = 0;
    int age = 0;
    int wage = 0;
};

struct Company
{
    Employee CEO;
    int cash = 0;
};

int main()
{
    Company myCompany { 1, 2, 50 };

    // Print out the area
    std::cout << "The size of Employee is " << sizeof(Employee) << "\n";

    return 0;
}
```

В этом случае, если бы мы хотели узнать, какая зарплата у CEO (использовалось директор), то нам бы пришлось использовать оператор выбора членов дважды:

```
Employee ceo{ 1, 42, 60.0 };
std::cout << ceo.boss.wage << '\n';
```

Согласно мы выбираем поле `CEO` из структуры `myCompany`, а затем поле `salary` из структуры `Employee`.

Вы можете использовать вложенные списки инициализации с вложенными структурами:

```
struct Employee
{
    int id = 0;
    int age = 0;
    int wage = 0;
};

struct Company
{
    Employee CEO;
    int cash = 0;
};

int main()
{
    Company myCompany { 1, 2, 50 };

    // Print out the area
    std::cout << "The size of Employee is " << sizeof(Employee) << "\n";

    return 0;
}
```

Размер структуры

Как правило, размер структуры — это сумма размеров всех ее членов, но не всегда! Например, рассмотрим структуру `Employee`. На большинстве платформ **two short** занимает 2 байта, тогда как 4 байта, а **two double** — 8 байт. Следовательно, ожидать, что `Employee` будет занимать 2 + 4 + 8 = 14 байт. Чтобы узнать точный размер `Employee`, мы можем воспользоваться **оператором sizeof**:

```
struct Employee
{
    int id = 0;
    int age = 0;
    double wage = 0.0;
};

int main()
{
    std::cout << "The size of Employee is " << sizeof(Employee) << "\n";

    return 0;
}
```

Результат выполнения программы (на компьютере автора):

```
The size of Employee is 16
```

Оказывается, мы можем сказать только, что размер структуры, по крайней мере, не меньше суммы размеров всех ее членов. Но он может быть и больше! По соображениям производительности компилятор иногда может добавлять отобразы! Промежутки в структуре.

В случае структуры `Employee` компилятор можно добавить 2 байта после члена `id`, увеличивая размер структуры до 16 байтов вместо 14. Отличное значение, но которой это происходит, выходит за рамки этого урока, но если вы хотите знать больше, то можете прочитать о **выравнивании данных в Википедии**.

Доступ к структурам из нескольких файлов

Поскольку объявления структуры не приводящих выделение памяти, то использовать **предварительные объявления** для них не сложно. Но есть обходной путь, если вы хотите использовать объявление структуры в нескольких файлах (чтобы иметь возможность создавать переменные этой структуры в нескольких файлах), поместите объявление структуры в **заголовочный файл** и подключайте этот файл везде, где необходимо использовать структуру.

Переменные типа struct подчиняется тем же правилам, что и обычные переменные. Следовательно, если вы хотите сделать переменные структуры доступны в нескольких файлах, то вы можете использовать **ключевое слово extern**.

Заключение

Структуры очень важны в языке C++, поскольку он поможет — это первый большой шаг на пути к объектно-ориентированному программированию! Чуть позже мы рассмотрим другой пользовательский тип данных — класс (который является продолжением структуры).

Тест

Задание №1

У вас есть веб-сайт и вы хотите отслеживать, сколько денег вы зарабатываете в день, от рекламодателей на нем рекламы. Объявите структуру `AdvertisingEarnings`, которая будет отслеживать:

- сколько объявлений вы показали посетителям (1);
- сколько пользователей посетителей нажали на объявления (2);
- сколько вы заработали в среднем за каждое нажатие на объявление (3).

Значения этих трех полей должны быть инициализированы нулями. Напишите функцию `displayEarnings` в функции, которая выводит каждую из этих значений, а также подсчитает, сколько всего денег вы заработали за день (перемножили все 3 поля).

Ответ №1

```
#include <iomanip> // for setprecision
#include <iostream>

struct AdvertisingEarnings
{
    int adsShown = 0;
    int clicksThroughAdsPercentage = 0;
    double averageEarningPerClick = 0.0;
};

void printEarnings(AdvertisingEarnings ae)
{
    // Calculate total earnings
    // Assumes all ads shown are clicked on
    // Assumes ad earnings are independent of each other
    double totalEarnings = ae.adsShown * (ae.clicksThroughAdsPercentage / 100.0) * ae.averageEarningPerClick;

    // Print the results
    // Use fixed notation and two decimal places when printing monetary values
    std::cout << "Ads shown: " << ae.adsShown << '\n';
    std::cout << "Ad clicks through percentage: " << ae.clicksThroughAdsPercentage << '%' << '\n';
    std::cout << "Average earnings per click: $" << ae.averageEarningPerClick << '\n';
    std::cout << "Total earnings: $" << totalEarnings << '\n';
}

int main()
{
    // Define variables to hold ad information
    AdvertisingEarnings ae;

    // Get ad shown
    std::cout << "How many ads shown today? ";
    std::cin >> ae.adsShown;

    // Get ad clicks through percentage
    std::cout << "What was the clicks through percentage? ";
    std::cin >> ae.clicksThroughAdsPercentage;

    // Get average earnings per click
    std::cout << "What was the average earnings per click? ";
    std::cin >> ae.averageEarningPerClick;

    // Print the results
    printEarnings(ae);

    return 0;
}
```

Задание №2

Создайте структуру для хранения дробных чисел. Структура должна иметь 2 члена: целочисленный числитель и целочисленный знаменатель. Объявите для дробных переменных в структуре по значению и пользователю. Напишите функцию **printFraction** которой будет при для переменной, которая будет переносить это число и выводить результат в виде дробного числа.

Ответ №2

```
struct Fraction
{
    int numerator;
    int denominator;
};

void setFraction(Fraction &f, int n1)
{
    // Assign the numerator
    f.numerator = n1;

    // Ask user for denominator
    // Declare a temporary variable to hold the denominator
    int temp;
    std::cout << "Enter a value for the denominator: ";
    std::cin >> temp;

    // Assign the denominator
    f.denominator = temp;
}

void printFraction(Fraction f)
{
    std::cout << "The fraction is: " << f.numerator << '/' << f.denominator << '\n';
}

int main()
{
    // Define variables to hold fractions
    Fraction f1;
    Fraction f2;

    // Get fraction 1
    setFraction(f1, 1);

    // Get fraction 2
    setFraction(f2, 2);

    // Print the fractions
    printFraction(f1);
    printFraction(f2);

    return 0;
}
```